

AI 협업 개발 보고서

CaseBoard

범죄 수사 보드 스타일 UI를 활용한
그래프 기반 프로젝트 관리 웹 서비스

프로젝트 CaseBoard

학번 3201

이름 박세희

작성일 2026-06-15



인공지능과 함께한 설계 · 구현 · 테스트 보고서

프로젝트: CaseBoard — 범죄 수사 보드 스타일 UI를 활용한 그래프 기반 프로젝트 관리 웹 서비스

유형: 개인 프로젝트 · 작성일: 2026-06-15

1. 개요

본 문서는 CaseBoard를 개발하며 AI 코딩 어시스턴트를 설계 · 구현 · 테스트 전 단계에 어떻게 활용했는지를 정리한 보고서입니다. AI는 코드를 대신 써 주는 도구를 넘어, 설계 의사결정의 토론 상대이자 버그를 함께 추적하는 페어 프로그래머로 활용했습니다.

서비스 한 줄 소개 — 사용자가 보드를 만들고 작업(Task)·버그(Bug)·사람(Person)·일정(Event)을 단서 카드처럼 배치한 뒤, 관계선으로 연결해 프로젝트 요소 간 흐름과 의존 관계를 시각적으로 관리하는 서비스.

기술 스택

구분	기술
프레임워크	Next.js 16 (App Router)
언어	TypeScript
ORM / DB	Prisma + PostgreSQL
인증	Auth.js (NextAuth) Credentials
그래프 UI	React Flow (@xyflow/react), dagre
스타일	Tailwind CSS
국제화	next-intl (한국어 / 영어)
배포	Vercel, Docker Compose (로컬 DB)

💡 AI 활용 원칙

- AI가 제안하면 사람이 검증한다 — 모든 산출물은 빌드·타입체크·동작 확인 후 반영
- "왜"를 항상 물었다 — 선택의 근거(트레이드오프)를 함께 설명하도록 요청
- 협업 기록을 남겼다 — AI가 함께 쓴 커밋에 **Co-Authored-By** 트레일러 표기

2. 설계 단계에서의 AI 활용

2.1 도메인 모델 설계

"수사 보드"라는 은유를 실제 데이터 모델로 환원. 핵심 논점은 "노드 타입을 어떻게 표현할 것인가".

- 검토 대안: (a) 자유 텍스트 태그 (b) 별도 테이블 분리 (c) Enum 고정 타입
- 결론: 타입이 4종으로 한정적이고 색상·분석 규칙이 타입에 강하게 결합되므로 Prisma `enum NodeType` 채택 → 마이그레이션 안정성 + 타입 안전성 동시 확보

```
enum NodeType { TASK  BUG  PERSON  EVENT }
```

```
model Board { id, title, description, ownerId, nodes[], edges[] ... }
```

```
model CaseNode { id, type: NodeType, title, content, x, y, boardId ... }
```

```
model CaseEdge { id, sourceNodeId, targetNodeId, label, boardId ... }
```

2.2 노드 위치 저장 방식의 분리 설계

드래그 좌표는 자주·빠르게 갱신되고, CRUD는 폼 기반의 무거운 작업이라는 점에 주목해 책임을 분리.

- 노드 좌표 갱신 → Route Handler `PUT /api/boards/[boardId]/nodes/[nodeId]/position` (리렌더 없이 좌표만 영속화)
- 보드/노드/관계선 CRUD → Server Action (`board-actions.ts`) → `revalidatePath` 로 일관 갱신

→ "Create/Update/Delete"와 "Server Action 및 Route Handler 사용" 제출 요건을 동시에 만족.

2.3 분석 알고리즘 설계

복잡한 그래프 알고리즘 대신 설명 가능한 차수(degree) 기반 휴리스틱 채택.

- 핵심 단서(critical): 연결 차수 ≥ 3
- 미연결 단서(isolated): 연결 차수 = 0
- 의심 관계 제안: 한 노드의 제목·내용에 다른 노드의 제목이 포함되면 후보 제안 (기존 연결 제외, 최대 5건)

💡 설계 근거: 시연 시 "왜 이게 핵심 단서인가"를 한 문장으로 설명할 수 있어야 한다는 요구를 AI와의 토론에서 도출 → 블랙박스 점수 대신 차수 규칙 채택.

3. 구현 단계에서의 AI 활용

3.1 인증 및 접근 제어

- Auth.js Credentials + Prisma Adapter, 비밀번호 `bcryptjs` 해시(`password.ts`)
- 보호 라우트 그룹 (`protected`) 레이아웃에서 세션 확인 → 비로그인 차단
- 소유권 검증을 쿼리 레벨에서 강제 → 다른 사용자의 노드 조작 불가

```
const result = await prisma.caseNode.updateMany({
  where: { id: nodeId, boardId, board: { ownerId: user.id } },
  data: { x: body.x, y: body.y }
});
if (result.count === 0) {
  return NextResponse.json({message: "Not found"}, {status: 404});
}
```

💡 **AI 협업 포인트:** 처음엔 `findUnique` 후 코드로 소유권을 비교했으나, AI가 "조건을 쿼리에 합치면 권한 없는 접근과 존재하지 않는 리소스를 동일하게 404로 처리해 정보 노출도 줄어든다"고 제안 → `updateMany` + `where` 방식으로 개선.

3.2 입력 검증 유틸리티

반복 패턴을 순수 함수로 분리(`validation.ts`)해 Server Action 곳곳에서 재사용.

```
readString(formData, key)    // FormData → trim된 문자열
normalizeEmail(email)        // 소문자 + trim
isValidEmail(email)          // 기본 형식 검증
```

3.3 국제화(i18n)

next-intl로 한국어/영어 지원. 잘못된 locale도 안전하게 기본값(`ko`)으로 fallback. 로딩/에러/404 UI를 locale별 제공.

3.4 그래프 에디터

React Flow 기반 드래그 앤 드롭, 관계선 생성/선택 삭제, 타입별 색상 구분(`clue-node.tsx`). dagre로 자동 레이아웃 보 조.

4. 테스트 · 검증 단계에서의 AI 활용

별도 단위 테스트 대신 **정적 검증 + 빌드 + 실제 배포 환경 동작 확인**의 다층 검증 체계를 운용하고, 각 단계에서 AI가 실패 원인 분석을 도왔습니다.

4.1 검증 파이프라인

```
npm run lint          # ESLint 정적 분석
npx tsc --noEmit --pretty false  # 타입 전수 검사
npm run build          # Next.js 16 Turbopack 프로덕션 빌드
npm run prisma:deploy  # 마이그레이션 적용
```

4.2 AI와 함께 추적·해결한 대표 버그

사례 ① 빈 환경변수로 인한 전 페이지 크래시 (Invalid URL)

- 증상: Vercel 배포 후 모든 페이지가 `TypeError: Invalid URL` 로 죽음
- 근본 원인: `NEXT_PUBLIC_APP_URL` 이 설정은 됐으나 빈 문자열(`"`). `?? "..."` 는 `null` / `undefined` 만 걸러서 빈 문자열이 통과 → `new URL("")` 예외
- 해결: `??` → `||` 로 변경, `layout.tsx` · `robots.ts` · `sitemap.ts` 세 곳에 동일 적용

```
- const appUrl = process.env.NEXT_PUBLIC_APP_URL ?? "http://localhost:3000";
+ const appUrl = process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000";
```

💡 학습 포인트: `??` (nullish)와 `||` (falsy)의 차이가 실제 운영 장애로 이어짐을 체크. 로컬에선 변수가 채워져 재현 안 되던 버그를 배포 환경 기준으로 분석.

사례 ② 프레임워크 정보 노출 차단

`next.config.ts` 에 `poweredByHeader: false` → `X-Powered-By` 헤더 제거, 공격 표면 축소.

사례 ③ 배포 자동화

`vercel.json` 빌드 단계에서 Prisma 마이그레이션 실행 → DB 스키마와 코드 불일치 예방.

4.3 수동 동작 확인 시나리오

#	시나리오	기대 결과
1	회원가입 → 로그인 → 로그아웃	세션 생성/파기 정상
2	비로그인으로 <code>/boards</code> 접근	로그인으로 차단
3	보드 생성 / 검색 / 삭제	목록 즉시 반영
4	노드 4종 생성 후 색상 구분	타입별 색상 일치
5	노드 드래그 이동 후 새로고침	좌표 영속화
6	관계선 생성/삭제 후 자동 분석	핵심/미연결/제안 갱신
7	한국어 ↔ 영어 전환	라우팅/문구 정상
8	로딩/에러/404 화면	전용 UI 노출

5. 회고: AI 협업의 효과와 한계

효과

- 설계 대안 탐색 속도 — 모델링·책임 분리의 트레이드오프를 빠르게 비교
- 근본 원인 분석 — `??` vs `||` 처럼 놓치기 쉬운 런타임 함정을 환경 차이 관점에서 규명
- 일관성 — 같은 수정을 영향받는 모든 파일에 빠짐없이 전파

한계와 대응

- AI 제안이 항상 옳진 않아 빌드·타입체크·실제 동작 확인을 최종 게이트로 둬
- 배포 환경 특유 문제(빈 환경변수 등)는 로컬 미재현 → 실제 배포본 기준 검증 병행

협업 기록의 추적성

AI와 함께 쓴 커밋은 `Co-Authored-By` 트레일러로 표기해 git 히스토리에서 기여를 확인 가능.